



CONHECENDO A LINGUAGEM DE PROGRAMAÇÃO **PYTHON**

Ministrante: Everaldo J. B. G. de Pádua
II Semana de Computação, Engenharia e Computação

Python é uma linguagem reconhecida por ser muito **versátil**, com ênfase na legibilidade do código, e por ter uma abordagem que permite aos programadores desenvolverem algoritmos **melhores estruturados**.



Guido Van Rossum
Criador do Python

VANTAGENS DO USO

- **Fácil** aprendizado;
- **Simples** de programar;
- Sintaxe **intuitiva**;
- Open Source;
- Modularização;
- Multiplataforma;
- Grande quantidade de bibliotecas disponíveis;
- Grande comunidade de usuários;
- Documentação extensa;
- Número de oportunidades no **mercado de trabalho**.

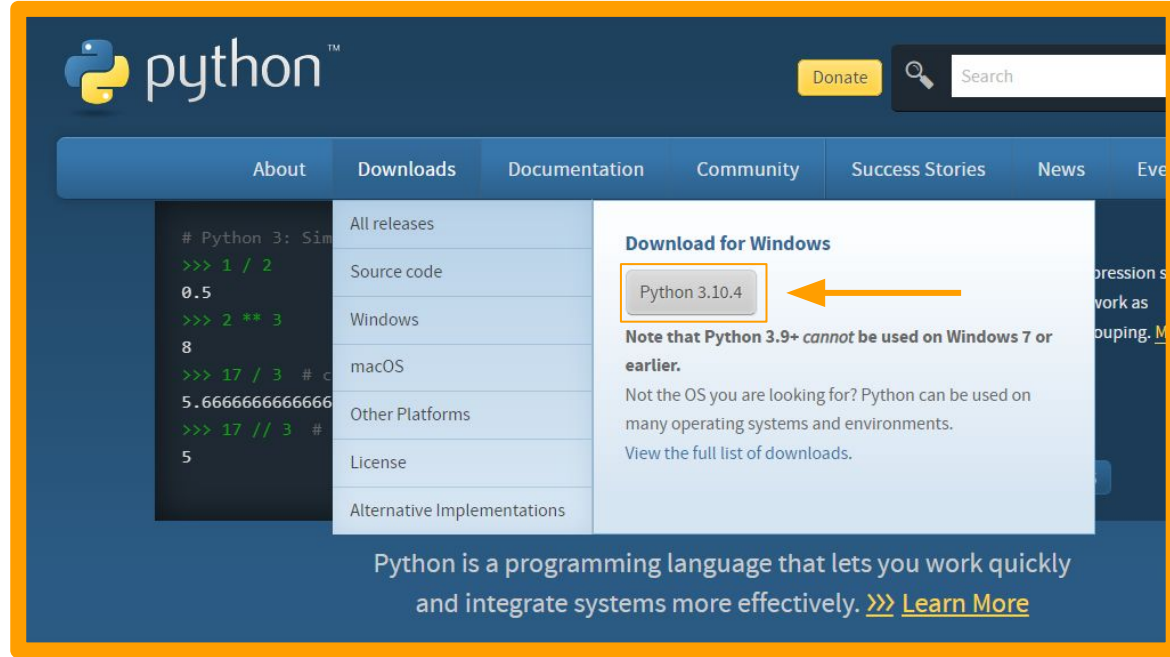
Conhecimentos de python mais requisitados no mercado:

- Testes com **Selenium**;
- Análise de Dados com **Pandas**;
- Desenvolvimento Web com **Django** e **Flask**;
- Inteligência Artificial com **Keras / Tensorflow / PyTorch**;
- GUI com **Tkinter / PyQT5**



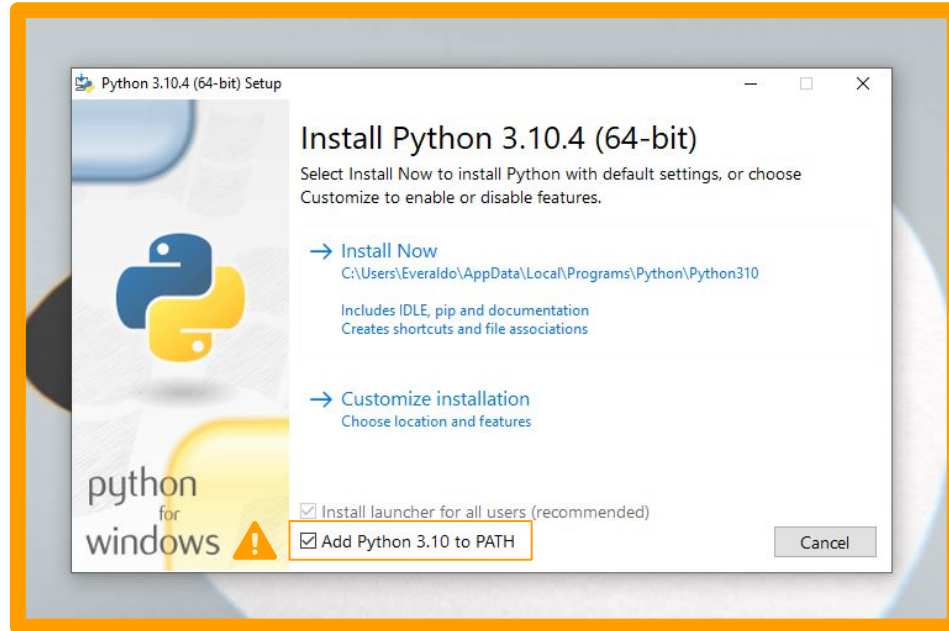


INSTALAÇÃO DO PYTHON



Você pode baixar a última versão do python pelo [Site Oficial](#)

No Windows



Adicione o python ao PATH do sistema, e clique em “instalar agora”

Atualmente, a linguagem já vem instalado por padrão.

Para verificar, abra o terminal e digite:

```
> which python3
```

Caso não apareça nada:

```
> sudo apt-get install python3
```

```
> sudo apt-get install python3-pip (opcional)
```

Não se esqueça de adicioná-lo ao PATH do sistema:

```
> export PATH="$PATH:/usr/local/bin/python3"
```

Onde posso compilar meus códigos?

IDEs

Visual Studio Code, Atom, Apache Netbeans, Pycharm, Notepad++, entre vários outros...

Compiladores online

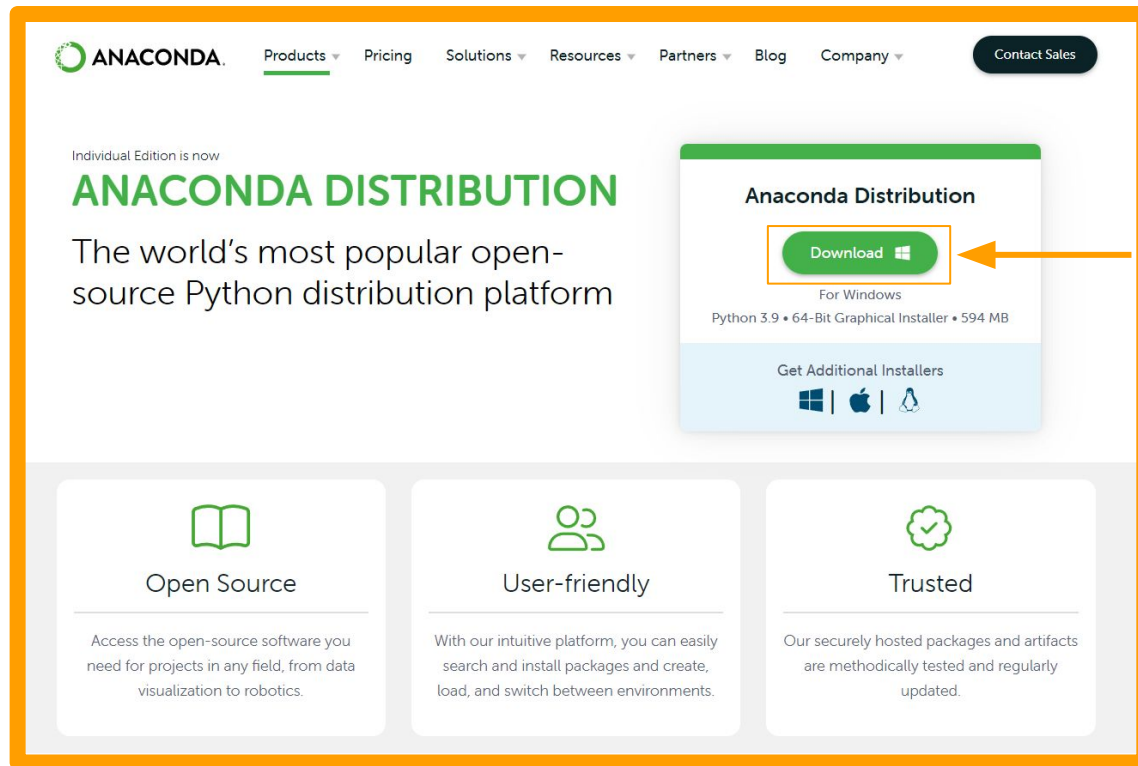
OnlineGDB, Replit, Programiz, entre outros...







Instalação do Jupyter Notebook



The screenshot shows the Anaconda website with a navigation bar at the top containing links for Products, Pricing, Solutions, Resources, Partners, Blog, Company, and a Contact Sales button. The main heading reads "Individual Edition is now ANACONDA DISTRIBUTION" followed by the tagline "The world's most popular open-source Python distribution platform". On the right, a card titled "Anaconda Distribution" features a green "Download" button with a Windows icon, which is highlighted by an orange rectangle and an arrow. Below the button, it specifies "For Windows" and "Python 3.9 • 64-Bit Graphical Installer • 594 MB". A section titled "Get Additional Installers" shows icons for Windows, macOS, and Linux. At the bottom, three feature boxes are displayed: "Open Source" (with an open book icon), "User-friendly" (with a person icon), and "Trusted" (with a checkmark icon), each accompanied by a brief description of its benefits.

Individual Edition is now

ANACONDA DISTRIBUTION

The world's most popular open-source Python distribution platform

Anaconda Distribution

[Download](#) 

For Windows
Python 3.9 • 64-Bit Graphical Installer • 594 MB

Get Additional Installers

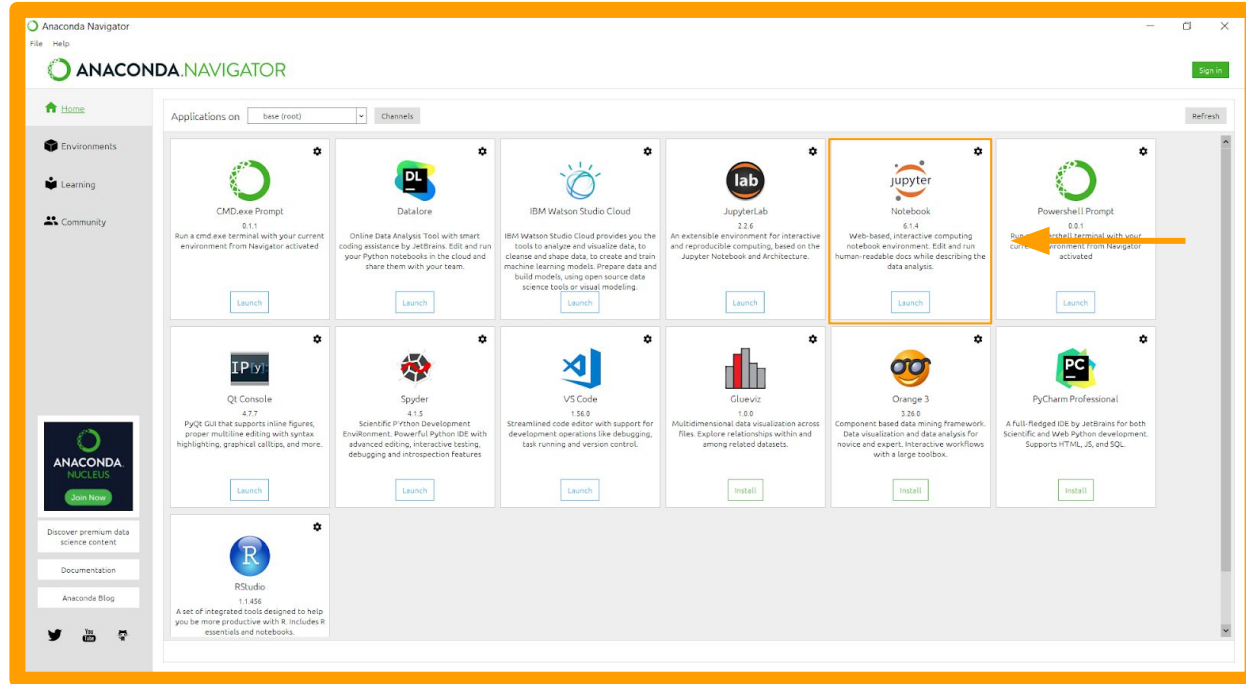

Open Source
Access the open-source software you need for projects in any field, from data visualization to robotics.


User-friendly
With our intuitive platform, you can easily search and install packages and create, load, and switch between environments.


Trusted
Our securely hosted packages and artifacts are methodically tested and regularly updated.

Você pode baixar o Anaconda pelo [Site Oficial](#)

No Windows



Entre as várias ferramentas do **Anaconda Navigator**, para executar a ferramenta vá em “Notebook”

Abre o terminal e execute os dois comandos:

```
> sudo apt-get install python3-pip
```

```
> sudo apt install jupyter-notebook
```

Para iniciar a ferramenta:

```
> jupyter-notebook
```

Primeiros Passos

Entrada e saída, variáveis,
operadores aritméticos

```
In [1]: print("Olá Mundo")
```

Olá Mundo

```
In [15]: print("Hora", 16, 20, sep=':')
```

Hora:16:20

```
In [12]: print("Everaldo", end=' ')  
print("Junior")
```

Everaldo Junior

Variáveis são pequenos espaços de memória, utilizados para **armazenar** e **manipular** dados.

Os principais tipos de variáveis são: *str*, *int* e *float*.

```
In [5]: texto = "Everaldo Junior"
        type(texto)
```

```
Out[5]: str
```

```
In [13]: inteiro = 10
         type(inteiro)
```

```
Out[13]: int
```

```
In [14]: flutuante = 3.14159
         type(flutuante)
```

```
Out[14]: float
```

Exemplos de declaração de variáveis

Nome	Válido	Comentários
2variavel	Não	Nomes de variáveis não podem começar com números
variavel2	Sim	Embora contenha um número, o nome a3 inicia com letra
nome sobrenome	Não	Nomes de variáveis não podem conter espaços em branco
nome_sobrenome	Sim	O símbolo (_) é permitido e facilita a leitura de nomes grandes

Palavras reservadas pelo python

False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

Elas não podem serem utilizadas como nome de variáveis

Operador	Descrição
+	Adição
-	Subtração
*	Multiplicação
/	Divisão
%	Resto
**	Potenciação
//	Divisão Inteira

```
In [27]: a, b = 5, 2
print("Adição:", a + b)
print("Diferença:", a - b)
print("Multiplicação:", a * b)
print("Potenciação:", a ** b)
print("Divisão:", a / b)
print("Divisão Inteira:", a // b)
print("Resto da divisão:", a % b)
```

```
Adição: 7
Diferença: 3
Multiplicação: 10
Potenciação: 25
Divisão: 2.5
Divisão Inteira: 2
Resto da divisão: 1
```

A função `input()` recebe dados em texto do usuário.

```
In [29]: nome = input("Digite seu nome: ")  
         print(nome)
```

```
Digite seu nome: Everaldo  
Everaldo
```

Caso queira receber outros tipos de dados, não se esqueça de fazer a **conversão**:

```
In [34]: numero = input("Digite um numero: ")  
         print(type(numero))  
         print(numero + numero)  
         numero = int(numero)  
         print(type(numero))  
         print(numero + numero)
```

```
Digite um numero: 2  
<class 'str'>  
22  
<class 'int'>  
4
```

2

Markdown

formatação de texto na web

Selecione o bloco de comandos, e, assim como na imagem, clique em *Markdown*.



Atalho do teclado para *markdown*: Tecla M

Títulos

```
# Título 1
## Título 2
### Título 3
#### Título 4
##### Título 5
```



Título 1

Título 2

Título 3

Título 4

Título 5

Lista

```
1. Item 1
2. Item 2
3. Item 3
```

```
* Item 1
  - item 1.1
  - item 1.2
* Item 2
  - item 2.1
  - item 2.2
* Item 3
  - item 3.1
  - item 3.2
```

1. Item 1
2. Item 2
3. Item 3

- Item 1
 - item 1.1
 - item 1.2
- Item 2
 - item 2.1
 - item 2.2
- Item 3
 - item 3.1
 - item 3.2

Ênfase e Links

1. ****negrito****
2. *_italico_*
3. [[Google](https://www.google.com)](https://www.google.com)

1. **negrito**
2. *italico*
3. [Google](#)

Citações

```
> "Eduquem as crianças e não será necessário castigar seus homens"  
>> Pitágoras
```

"Eduquem as crianças e não será necessário castigar seus homens"

Pitágoras

Separadores



Cores

```
<font color=green>Verde</font>  
<font color=blue>Azul</font>  
<font color=purple>Roxo</font>
```

Verde

Azul

Roxo

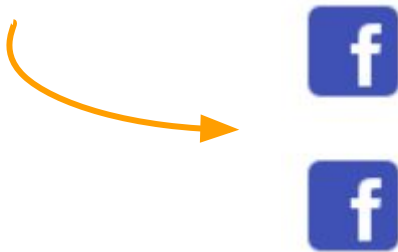
Quebra de linha: 2 espaços e enter.

O Atributo **color** também aceita hexadecimais

Imagens

```
![facebook](facebook.png)  

```



Código

```
~~~py  
num1 = int(input("Digite o primeiro numero: "))  
num2 = int(input("Digite o segundo numero: "))  
print("Resultado da multiplicação:", num1 * num2)  
~~~
```

```
num1 = int(input("Digite o primeiro numero: "))  
num2 = int(input("Digite o segundo numero: "))  
print("Resultado da multiplicação:", num1 * num2)
```

Sugestão de site para ícones: [Icons8](#)

Tabelas

Produto	Preço(R\$)	Validade
:-----:	:-----:	:-----:
Produto 1	10,00	29/05/2022
Produto 2	9,89	02/06/2022
Produto 3	24,49	12/03/2023
Produto 4	2,50	01/01/2023

Produto	Preço(R\$)	Validade
Produto 1	10,00	29/05/2022
Produto 2	9,89	02/06/2022
Produto 3	24,49	12/03/2023
Produto 4	2,50	01/01/2023

- **Alinhado a esquerda:**
usar ‘:’ no lado esquerdo
(alinhamento padrão);
- **Alinhado a direita:**
usar ‘.’ no lado direito;
- **Centralizado:**
usar ‘.’ dos dois lados.

Equações

```

$$
\sum_{n=1}^{\infty} \frac{2^n}{n!} =
\lim_{n \rightarrow \infty} \frac{2^{n+1}}{(n+1)!} \cdot \frac{n!}{2^n}
$$

```

$$\sum_{n=1}^{\infty} \frac{2^n}{n!} = \lim_{n \rightarrow \infty} \frac{2^{n+1}}{(n+1)!} \cdot \frac{n!}{2^n}$$

Sugestão de site para montar equações: [Online Latex Editor](#)



Exercício 1: Faça a equação abaixo em markdown e o implemente em python.

$$c = \frac{a^2 + b^2}{(a - b)^2}$$

**Receba os valores pelo usuário, e imprima o resultado.*

ESTRUTURA DE CONDIÇÃO

Se?... E se?... se não...

Tudo que estiver dentro da **tabulação**, será um bloco de comando

```
if condicao:
    ...
    ...
    ...
elif condicao2:
    ...
    ...
    ...
else:
    ...
    ...
    ...
```

- **if:** executa se a condição for verdadeira(**True**);
- **elif:** para quando a estrutura tiver mais de uma condição;
- **else:** quando todas condições forem falsas(**False**);

Atalho do teclado para tabulação: Tecla TAB

Operadores Condicionais

Operador	Descrição
<	Menor do que
<=	Menor ou Igual do que
>	Maior do que
>=	Maior ou igual do que
==	Igual a
!=	Diferente de

Operadores Lógicos

Operador	Descrição
and	Todas condições necessitam serem verdadeiras
or	Pelo menos uma condição deve ser verdadeira
not	Inverte o valor da condição
is	Verdadeiro se os identificadores das condições forem os mesmos



Exercício 2: Faça um programa para o cálculo de uma folha de pagamento, sabendo que os descontos são do Imposto de Renda, que depende do salário bruto e 10% para o INSS. O Salário Líquido corresponde ao Salário Bruto menos os descontos. O programa deverá pedir ao usuário o valor da sua hora e a quantidade de horas trabalhadas no mês.

Desconto do IR:

Salário Bruto até 900 (inclusive) - isento;

Salário Bruto até 1500 (inclusive) - desconto de 5%;

Salário Bruto até 2500 (inclusive) - desconto de 10%;

Salário Bruto acima de 2500 - desconto de 20%.

ESTRUTURA DE REPETIÇÃO
E se eu precisar repetir uma tarefa mais de uma
vez?

while: o bloco de comando será executado enquanto a condição for verdadeira.

```
In [77]: while condicao:
```

```
    ...  
    ...  
    ...
```

for: usado para repetir um intervalo de valores.

```
In [ ]: for valor in range(valor_minimo, valor_maximo):
```

```
    ...  
    ...  
    ...
```

while:

```
In [80]: i = 0
        while i < 10:
            print(i)
            i = i + 1
```

0
1
2
3
4
5
6
7
8
9

for:

```
In [81]: for i in range(0, 10):
        print(i)
```

0
1
2
3
4
5
6
7
8
9

A função range() ignora o último valor.



Exercício 3: Defina uma senha em uma variável qualquer, e solicite o usuário para adivinhar a senha. Faça isso enquanto o usuário não acertar.

4

Listas (*list*)
como lidamos com um conjunto de valores ou
variáveis?

Existem várias maneiras de se criar uma nova lista. A maneira mais simples é envolver os elementos da lista por **colchetes** ([e]).

Para buscarmos determinado valor, usamos o colchetes e passamos seu **índice**:

```
In [6]: ['Carro', 'Moto', 'Bicicleta']  
primos = [1, 2, 3, 5, 7, 11]  
print(primos[0])  
print(primos[5])
```

1

11

A primeira posição da lista sempre será 0.

FATIAMENTO DE LISTA

Passando “:” dentro do colchetes, podemos fatiar uma lista:

```
In [13]: print(primos[:])  
         print(primos[1:])  
         print(primos[:2])  
  
         [1, 2, 3, 5, 7, 11]  
         [2, 3, 5, 7, 11]  
         [1, 2]
```

O elemento no primeiro parâmetro sempre será incluso no fatiamento, enquanto o segundo parâmetro não será.

OPERAÇÕES

Podemos concatenar (+) e repetir (*) listas:

```
In [40]: print([0, 1] + [0, 1] + [0, 1])  
         print([0, 1] * 3)
```

```
[0, 1, 0, 1, 0, 1]
```

```
[0, 1, 0, 1, 0, 1]
```

TUPLAS (*Tuple*)

As tuplas, assim como a lista, são um conjunto de valores, sua diferença é que seus dados são **imutáveis** e sua declaração é através de parênteses ((e)).

```
In [84]: numeros = (1, 2, 3, 4)
         print(numeros)
         print(numeros[3])
         numeros[3] = 5
```

```
(1, 2, 3, 4)
4
```

```
-----
TypeError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_7092\2541032112.py in <module>
      2 print(numeros)
      3 print(numeros[3])
----> 4 numeros[3] = 5

TypeError: 'tuple' object does not support item assignment
```

Com as tuplas podemos atribuir valores a um conjunto de variáveis, essa técnica é chamada de “**desempacotamento**”. Dessa forma, podemos fazer operações:

```
In [40]: a, b, c, d = numeros  
         print(a + b, c * d)
```

```
3 12
```

Método	Descrição
<code>len</code>	retorna o tamanho da lista
<code>min</code>	retorna o menor valor da lista
<code>max</code>	retorna o maior valor da lista
<code>append</code>	retorna soma dos elementos da lista
<code>extend</code>	adiciona um novo valor na no final da lista
<code>del</code>	insere uma lista no final de outra lista
<code>in</code>	verifica se um valor pertence à lista.
<code>sort()</code>	ordenar em ordem crescente
<code>reverse()</code>	inverte os elementos de uma lista.

ITERAÇÃO DE LISTA

Também podemos usar a estrutura de repetição “for” para percorrer uma lista.

```
In [95]: comidas = ["Pizza", "Hamburguer", "Chocolate", "Sopa"]  
         for comida in comidas:  
             print(comida)
```

```
Pizza  
Hamburguer  
Chocolate  
Sopa
```



Exercício 4: Armazene os nomes e as notas de 5 alunos em listas, e imprima:

1. Os alunos que ficaram como nota abaixo de 6;
2. O aluno com a maior nota;
3. A média de todas as notas.

STRING

Todo texto em programação, é um conjunto de caracteres, logo podemos acessar seus **elementos** como numa lista:

```
In [35]: nome = "Everaldo Junior Borges"
          print(nome[0])
          print(nome[:8])
```

```
E
Everaldo
```

Método	Descrição
<code>count()</code>	Informa quantas vezes um caractere aparece na string
<code>capitalize()</code>	Retorna a string com a primeira letra maiúscula
<code>isalnum()</code>	Verifica se a string possui algum conteúdo alfanumérico (letra ou número)
<code>isalpha()</code>	Verifica se a string possui apenas conteúdo alfabético
<code>upper()</code>	Retorna uma cópia da string trocando todas as letras para maiúsculo
<code>split()</code>	Transforma a string em uma lista, utilizando os espaços como referência
<code>replace(S1, S2)</code>	Substitui na string o trecho S1 pelo trecho S2
<code>endswith()</code>	Verifica se uma string termina com uma determinada sequência

Mais métodos em: [Python String Methods](#)



Exercício 5: Peça ao usuário para digitar um email e senha, e repita a operação caso não respeite as seguintes validações:

Para o email:

1. Todas as letras devem ser minúsculas
2. Deve terminar com '@gsuite.iff.edu.br'

Para a senha:

3. Contenha mais do que 6 caracteres;
4. Não possua nenhum espaço em branco;

```
In [93]: conjunto = [-2, -1, 0, 1, 2]
         novo_conjunto = []
         for x in conjunto:
             if x > 0:
                 novo_conjunto = novo_conjunto + [x * 2]
         novo_conjunto
```

Out[93]: [2, 4]

```
In [86]: conjunto = [-2, -1, 0, 1, 2]
         [x * 2 for x in conjunto if x > 0]
```

Out[86]: [2, 4]

Dicionário (*dict*)
como lidamos com um conjunto de valores ou
variáveis?

Dicionário é um conjunto de valores, onde cada valor é associado a uma chave de acesso.

```
diccionario = {  
    'chave_1': 'valor_1',  
    'chave_2': 'valor_2',  
    'chave_3': 'valor_3',  
    ...  
    'chave_n': 'valor_n',  
}
```

Para acessar um determinado elemento, assim como vetor, invés de passar o índice, você deve passar a **chave** do elemento.

```
In [106]: cardapio = {  
    "Hamburger": 9,  
    "Salgado Assado": 5,  
    "Refrigerante": 3,  
    "Suco": 1.5,  
    "Doce": 2  
}  
print(cardapio)  
print(cardapio['Suco'], cardapio.get('Hamburger'), sep=', ')
```

{'Hamburger': 9, 'Salgado Assado': 5, 'Refrigerante': 3, 'Suco': 1.5, 'Doce': 2}
1.5, 9

Você pode alterar valores através de sua chave, caso o elemento do *dict* não exista, ele será **acrescentado**.

```
In [107]: cardapio["Suco"] = 2  
cardapio["Salgado Frito"] = 4.5  
cardapio
```

```
Out[107]: {'Hamburger': 9,  
           'Salgado Assado': 5,  
           'Refrigerante': 3,  
           'Suco': 2,  
           'Doce': 2,  
           'Salgado Frito': 4.5}
```



Exercício 6: Armazene os nomes e as notas de 5 alunos em **dicionário**, e imprima:

1. Os alunos que ficaram como nota abaixo de 6;
2. O aluno com a maior nota;
3. A média de todas as notas.

6

Funções nomeando tarefas no código

Uma **função** é uma sequência de comandos que executa alguma tarefa e que tem um nome.

```
In [112]: def nome_funcao(parametros):  
          ...  
          ...
```

```
In [115]: def soma_valores(num1, num2):  
          print("Soma:", num1 + num2)
```

```
In [116]: a, b = (2, 3)  
          soma_valores(a, b)
```

Soma: 5

Funções podem **retornar** qualquer tipo de valor:

```
In [118]: def soma_valores(num1, num2):  
          return num1 + num2
```

```
In [119]: a, b = (2, 3)  
          print("Soma:", soma_valores(a, b))
```

Soma: 5

Com um texto entre três aspas ("""), você pode definir uma descrição para a função:

```
In [120]: def soma_valores(num1, num2):  
          '''Retorna a soma entre dois elementos'''  
          return num1 + num2
```

```
In [121]: a, b = (2, 3)  
          print("Soma:", soma_valores(a, b))
```

In [61]:

Signature: soma_valores(num1, num2)

Docstring: Retorna a soma entre dois elementos

File: c:\users\everaldo\appdata\local\temp\ipykernel_7092\951530272.py

Type: function

Atalho do teclado para ver a documentação da função: Shift + TAB

Com `*` antes de alguma variável do parâmetro, você pode receber uma tupla de valores:

```
In [5]: def f(*args):  
        print(f'\nargs={args}')  
        for arg in args:  
            print(arg, end=' ')  
  
        f()  
        f(1)  
        f(2, 3)  
        f(4, 5, 6)  
  
        args=()  
  
        args=(1,)  
        1  
        args=(2, 3)  
        2 3  
        args=(4, 5, 6)  
        4 5 6
```

E passando **, você pode passar um dicionário com valores.

```
In [8]: def f(**kwargs):  
        print(f'\nargs={kwargs}')  
        for chave, valor in kwargs.items():  
            print(chave, valor)  
  
        f()  
        f(a=1)  
        f(b=2, c=3)  
        f(d=4, e=5, f=6)  
  
args={}  
  
args={'a': 1}  
a 1  
  
args={'b': 2, 'c': 3}  
b 2  
c 3  
  
args={'d': 4, 'e': 5, 'f': 6}  
d 4  
e 5  
f 6
```

Porém, no momento de chamar a função, ela deverá ter um nome atribuído para o valor chave.



Exercício 7: Crie uma função que receba o nome do usuário, e suas coisas favoritas, e imprima na tela seu nome e seus gostos em forma de lista.

Bibliotecas

funções já prontas disponíveis para uso

As bibliotecas foram criadas com intuito de **facilitar** a vida do desenvolvedor. Podemos implementá-las no código com a função **import**, e chamar suas funções da seguinte forma:

```
In [65]: import math
```

```
In [66]: math.sqrt(4)
```

```
Out[66]: 2.0
```

```
In [67]: math.cos(math.pi)
```

```
Out[67]: -1.0
```

Também podemos implementar funções específicas das bibliotecas com o comando **from**:

```
In [2]: from math import sin, cos
```

```
In [6]: sin(math.pi/2)
```

```
Out[6]: 1.0
```

```
In [7]: cos(math.pi)
```

```
Out[7]: -1.0
```

Por fim, podemos nomear nossas funções incorporadas, com o **as**:

```
In [10]: from math import log10 as logaritimo
```

```
In [12]: logaritimo(2)
```

```
Out[12]: 0.3010299956639812
```

Algumas bibliotecas utilizados no curso

Link para documentação
<u>Math</u>
<u>Numpy</u>
<u>Pandas</u>
<u>Mathplotlib</u>

Caso não tenha a biblioteca instalada:

```
> pip install <biblioteca>
```



Exercício 8: Uma professora lhe enviou vários arquivos contendo o nome e notas de cada um dos seus 30 alunos, nos 4 bimestres. Passe todos esses valores para um dicionário em python, e permite visualizar dados como:



1. Gráficos com as notas dos alunos por bimestre, em ordem decrescente;
2. Alunos com notas menores do que 6.0 no bimestre;
3. As 5 melhores notas do bimestre;
4. Uma tabela com todas as notas;
5. Acrescente uma coluna com a nota média de cada aluno;
6. Acrescente na tabela uma linha com a nota média de cada bimestre

OBRIGADO!

Alguma dúvida?

Por onde entrar em contato?

Email: junior.borges@gsuite.iff.edu.br

Linkedin: [Linkedin](#)

Insta: [@everaldojr13](#)

